

PROGRAMMATION EN PYTHON

Lors de l'écriture d'un programme, il est très important de commenter abondamment les lignes de codes afin de vous souvenir de leurs significations et de faciliter la compréhension de l'ensemble par des lecteurs éventuels. En Python, le symbole « # » précède les commentaires.

1. VARIABLES, AFFECTATION, ENTREES, SORTIES ET OPERATIONS

1.1. VARIABLES

Une variable est une zone de la mémoire dans laquelle une valeur est stockée. Aux yeux du programmeur, cette variable est définie par un nom, alors que pour l'ordinateur, il s'agit en fait d'une adresse (une zone particulière de la mémoire). En Python, la déclaration d'une variable et son affectation (c'est à dire la première valeur que l'on va stocker dedans) peut se faire en même temps.

Les noms de variables sont des noms que vous choisissez vous-même assez librement. Efforcez-vous cependant de bien les choisir : de préférence assez courts, mais aussi explicites que possible, de manière à exprimer clairement ce que la variable est censée contenir. N'utilisez les majuscules qu'à l'intérieur même du nom, pour en augmenter éventuellement la lisibilité (exemple : la variable « AltitudeDeLaMontagne » pourrait contenir comme son nom l'indique l'altitude de la montagne)

Il existe différents types de variables, permettant de manipuler les données :

Type	Signification	Taille (en octets)	Remarques
int	Entier	4 soit 32 bits	$-2^{31} \approx -2,147 \times 10^9 \rightarrow +2^{31} - 1 \approx 2,147 \times 10^9$
long	Entier long	8 soit 64 bits	$-2^{63} \approx -9,223 \times 10^{18} \rightarrow +2^{63} - 1 \approx 9,223 \times 10^{18}$
float	Nombre réel simple	4 soit 32 bits	$\pm 2^{-149} \approx 1.4 \times 10^{-45} \rightarrow \pm 2^{128} - 2^{104} \approx 3.4 \times 10^{38}$
complex	Nombre complexe	4 soit 32 bits	a + ib
boolean	Valeur logique (boolée)	1 soit 8 bits	true (vrai), ou false (faux)

1.2. AFFECTATIONS DE VARIABLES

La façon dont sera déclarée et affectée la variable permet de fixer son type :

Syntaxes	Actions
<code>nbr = 1</code>	Déclare et affecte 1 à la variable (nbr) de type entier (int)
<code>nbr = 12.9</code>	Déclare et affecte 12, 9 à la variable (nbr) de type réel (float)
<code>nbr1, nbr2, nbr3 = 1, 2.6, 0.75</code>	Déclare et affecte simultanément 1 à la variable (nbr1), 2, 6 à la variable (nbr2) et 0, 75 à la variable (nbr3). Ces variables sont de type réel (float)
<code>chaine = "bonjour"</code>	Déclare et affecte bonjour à la variable (chaine) de type chaîne de caractère (str)
<code>liste = [1, 3, "bonjour", 3.141]</code>	Déclare et affecte 1, 3, bonjour et 3.141 à la variable (liste) de type liste. Cette liste est modifiable.
<code>tuple = (1, 3, "bonjour", 3.141)</code>	Déclare et affecte 1, 3, bonjour et 3.141 à la variable (tuple) de type tuple ou p-uplet. Cette liste est non modifiable.

Syntaxes	Actions
<code>dictionnaire = {"civilité": "Monsieur", "nom": "Faye", "adresse": "6 rue Paul Dérignac, 87000 Limoges"}</code>	Déclare et affecte Monsieur pour la clé civilité , Faye pour la clé nom et 6 rue Paul Dérignac, 87000 Limoges pour la clé adresse de type dictionnaire. Il est modifiable.

1.3. ENTREES

Afin de pouvoir dialoguer avec un programme en cours d'exécution, il est parfois nécessaire de saisir une valeur que demande le programme.

Syntaxes	Actions
<code>nbr = float(input("Veuillez saisir un nombre"))</code>	Demande à l'utilisateur de saisir un nombre flottant et affecte (mémoire) la donnée saisie dans la variable nbr
<code>nbr = int(input("Veuillez saisir un nombre"))</code>	Demande à l'utilisateur de saisir un nombre entier et affecte (mémoire) la donnée saisie dans la variable nbr

Attention : `input()` est une fonction qui renvoie toujours une chaîne de caractères. Pour changer le type d'une variable, on utilise : `int()` pour les entiers, `float()` pour les nombres à virgule flottante ou `str()` pour les chaînes de caractères (inutile après la fonction `input()`).

1.4. SORTIES

Pour permettre au programme en cours d'exécution d'afficher un texte ou un nombre on utilise la fonction `print()` :

Syntaxes	Actions
<code>print("Bonjour tout le monde")</code>	Affiche le message : Bonjour tout le monde
<code>print(22)</code>	Affiche le message : 22
<code>print("Chat", "chien", sep=" et ", end=" !!!")</code>	Affiche le message : Chat et chien !!!
<code>a=2 print("Le carré de ", a, " est ", a*a)</code>	Affiche le message : Le carré de 2 est 4

1.5. PRINCIPALES OPERATIONS

Syntaxes	Actions
<code>a+b</code>	Addition de a et b ou concaténation pour des chaînes de caractères
<code>a-b</code>	Soustraction de b à a
<code>a*b</code>	Multiplication de a par b ou répétition pour des chaînes de caractères
<code>a**b</code> exemple <code>2**4</code> donne 16	Elévation de a à la puissance b
<code>Sqrt(a)</code>	Racine carrée de a
<code>a/ b</code> exemple <code>23/3</code> donne 7,666666666666667	Division de a par b
<code>a//b</code> exemple <code>22//5</code> donne 4	Division euclidienne (entière) de a par b
<code>a% b</code> exemple <code>22%5</code> donne 2	Reste de la division euclidienne de a par b

2. TESTS, INSTRUCTIONS, BOUCLES CONDITIONNELLES ET ITERATIVES

2.1. TESTS ET INSTRUCTIONS CONDITIONNELLES

if() :, **elif()** :, **else**: permettent de tester une ou plusieurs conditions (le contenu d'une ou plusieurs variables) et d'exécuter une série d'instructions si la ou les conditions sont remplies :

Syntaxes	Actions
<pre>if (condition): instructions</pre>	Teste la condition condition et exécute les instructions indentées instructions
<pre># exemple nbr = float(input("Veuillez saisir un nombre")) if (nbr == 10): print("vous avez saisi 10")</pre>	Demande à l'utilisateur de saisir un nombre flottant et affecte (mémoire) la donnée saisie dans la variable nbr . Si nbr est égale à 10 affiche le message vous avez saisi 10
Syntaxes	Actions
<pre>if (condition): instructions 1 else: instructions 2</pre>	Teste la condition condition puis exécute les instructions indentées instructions 1 si la condition est remplie ou exécute les instructions indentées instructions 2 si la condition n'est pas remplie
<pre># exemple nbr = float(input("Veuillez saisir un nombre")) if (nbr == 10): print("vous avez saisi 10") else: print("vous n'avez pas saisi 10")</pre>	Demande à l'utilisateur de saisir un nombre flottant et affecte (mémoire) la donnée saisie dans la variable nbr . Si nbr est égale à 10 affiche le message vous avez saisi 10 sinon affiche le message vous n'avez pas saisi 10
Syntaxes	Actions
<pre>if (condition 1): instructions 1 elif (condition 2): instructions 2 else: instructions 3</pre>	Teste la condition condition 1 . Si cette condition 1 est vérifiée exécute les instructions indentées instructions 1 , sinon teste la condition condition 2 . Si cette condition 2 est vérifiée exécute les instructions indentées instructions 2 , sinon exécute les instructions indentées instructions 3
<pre># exemple nbr = float(input("Veuillez saisir un nombre")) if (nbr < 0): print("votre nombre est négatif") elif (nbr==0): print("votre nombre est nul") else: print("votre nombre est positif") print("le programme est fini")</pre>	Demande à l'utilisateur de saisir un nombre flottant et affecte (mémoire) la donnée saisie dans la variable nbr . Si nbr est < 0 affiche le message votre nombre est négatif , sinon teste si nbr est = 0 et si cette condition est vérifiée affiche le message votre nombre est nul . Si aucunes des conditions précédentes sont vérifiées affiche le message votre nombre est positif . Dans tous les cas affiche le message le programme est fini

Conditions et opérateurs logiques (booléens) :

Syntaxes	Actions
a == b	Teste si la variable a est égale à la variable b
a < b	Teste si la variable a est inférieure à la variable b
a > b	Teste si la variable a est supérieure à la variable b
a <= b	Teste si la variable a est inférieure ou égale à la variable b
a >= b	Teste si la variable a est supérieure ou égale à la variable b
a != b	Teste si la variable a est différente de la variable b
(condition 1) or (condition 2) (a < 5) or (a > 10)	Teste si la variable a est inférieure à 5 ou supérieure à 10
(condition 1) and (condition 2) (a > 5) and (a < 10)	Teste si la variable a est supérieure à 5 et inférieure à 10 donc comprise entre 5 et 10 exclus
not (condition) not (a<=3)	Teste si la variable a n'est pas inférieure ou égale à 3 donc strictement supérieur à 3

2.2. BOUCLES CONDITIONNELLES ET ITERATIVES

2.2.1. BOUCLES CONDITIONNELLES NON BORNEES TANT QUE / WHILE

Syntaxes	Actions
while (condition): instructions	Exécute les instructions indentées instructions tant que condition condition est vérifiée
# exemple nbr = 0 while (nbr <= 10): print ("a vaut " + str (nbr)) nbr = nbr + 1 print ("Le programme est terminé")	Affecte 0 (mémoire) à la variable nbr . Tant que nbr est inférieure ou égale à 10, affiche le message a vaut suivi de la valeur actuelle de a (de 0 à 10). Lorsque a atteint la valeur 11, affiche le message Le programme est terminé

2.2.2. BOUCLES ITERATIVES BORNEES POUR / FOR

Syntaxes	Actions
for variable in range (nbr): instructions	Exécute en boucle les instructions indentées instructions pour la variable variable allant de 0 à nbr-1
# exemple for m in range (10): print (m , end = ", ") print ("\\n Le programme est terminé")	Affiche tous les nombres de 0 à 9 sur une même ligne puis le message Le programme est terminé sur la ligne suivante
for variable in range (nbrN , nbrM): instructions	Exécute en boucle les instructions indentées instructions pour la variable variable allant de nbrN à nbrM-1
# exemple for m in range (4 , 10): print (m , end = ", ") print ("\\n Le programme est terminé")	Affiche tous les nombres de 4 à 9 sur une même ligne puis le message Le programme est terminé sur la ligne suivante

Syntaxes	Actions
for variable in range (nbrN , nbrM , pas): instructions	Exécute en boucle les instructions indentées instructions pour la variable variable allant de nbrN à nbrM-1 avec un pas pas
# exemple for m in range (1 , 10 , 2): print (m , end = " ") print ("\\n Le programme est terminé")	Affiche tous les nombres impairs de 1 à 9 sur une même ligne puis le message Le programme est terminé sur la ligne suivante
# exemple avec une liste jourSemaine =["lundi", "mardi", "mercredi", "jeudi", "v", edi , "samedi", "dimanche"] for jour in jourSemaine : print (jour) print ("Le programme est terminé")	Affiche tous les jours de la semaine sur 7 lignes différentes puis le message Le programme est terminé sur la ligne suivante

3. FONCTIONS ET PROCEDURES

3.1. FONCTIONS

Une fonction est un programme qui a un nom, qui utilise aucun, un ou plusieurs paramètres (variables locales) et qui renvoie aucun, un ou plusieurs résultats (variables locales). Une fonction peut être réutilisée par une autre fonction ou un autre programme.

Syntaxes	Actions
def nomDeLaFonction (param1 , param2 = val): instructions return var1 , var2	Exécute la fonction nomDeLaFonction ayant pour paramètres param1 et param2 . Lors de l'exécution de cette fonction le paramètre param2 prendra la valeur val . A la fin de l'exécution cette fonction renverra les variables var1 et var2 .
# exemple fonction cube def cube (nbr): cubeNbr = nbr **3 return cubeNbr # Programme principal appel et utilisation de la fonction nbrSaisi = int (input ("Saisir un nombre entier")) resultat = cube (nbrSaisi) print (resultat)	Demande à l'utilisateur de saisir un nombre puis exécute la fonction cube ayant pour paramètre nbr . Cette fonction calcule et renvoie la variable cubeNbr correspondant au cube du nombre saisi. Le programme affiche le cube du nombre saisi.

3.2. PROCEDURES

Une procédure est une fonction qui ne renvoie pas de valeur.

Syntaxes	Actions
def nomDeLaProcédure (param1 , param2 = val): instructions	Exécute la procédure nomDeLaProcédure ayant pour paramètres param1 et param2 . Lors de l'exécution de cette procédure le paramètre param2 prendra la valeur val .
# exemple fonction table de multiplication def tableMultiplication (tableDe): nbr = 0 while (nbr <= 10): print (nbr , " x ", tableDe , " = ", nbr * tableDe) nbr = nbr + 1 # Programme principal appel et utilisation de la fonction nbrSaisi = int (input ("Saisir un nombre entier")) tableMultiplication (nbrSaisi)	Demande à l'utilisateur de saisir un nombre puis exécute la procédure tableMultiplication ayant pour paramètre tableDe . Cette procédure calcule et affiche les produits du nombre saisi par les nombres de 0 à 10.

4. GRAPHIQUES

4.1. GRAPHIQUES

Afin de représenter un graphique on doit importer les bibliothèques :

```
from matplotlib import*
```

ou

```
from pylab import*
```

Les instructions suivantes permettent de configurer une ligne comportant 5 points entre, d'une part, les abscisses xa et xb, et d'autre part, les ordonnées ya et yb :

```
x = linspace(xa,xb,5)
```

```
y = linspace(ya,yb,5)
```

Les instructions suivantes permettent de tracer le segment configuré ci-dessus d'une épaisseur de 2 et d'ajouter une légende :

```
plot(x,y,"k:o",label="légende",linewidth=2)
```

Les instructions suivantes permettent d'ajouter du texte à la position (2,4) :

```
plt.text(2,4,"texte")
```

Les instructions suivantes permettent d'ajouter les axes, un titre, une grille, la légende, les noms des axes et d'afficher l'ensemble :

```
plt.axis([-5,5,-10,10])
```

```
# tracer le titre
```

```
plt.title("Segment")
```

```
# tracer la grille
```

```
plt.grid()
```

```
# tracer la légende
```

```
plt.legend()
```

```
# tracer les noms des axes
```

```
xlabel("abscisses")
```

```
ylabel("ordonnees")
```

```
# afficher l'ensemble
```

```
show()
```

4.2. STYLE DE LIGNE

Les chaînes de caractères suivantes permettent de définir le style de ligne :

Chaînes	Effets
-	Ligne continue
--	Tirets
:	Ligne en pointillé
-. .	Tirets points
-o	Tirets gros points
k	Couleur noire
b	Couleur bleue
r	Couleur rouge
g	Couleur verte
color=(0.7,0.1,0.2)	Couleur RVB

5. **BIBLIOTHEQUES**

De nombreuses bibliothèques sont incluses dans Edupython :

- Module lycee
- Calcul numérique : numpy
- Sorties graphiques : matplotlib ou pylab
- Calcul formel : sympy
- Traitement d'images : PIL (Pillow)
- Liaison série : serial
- Tortue : turtle
- Base de données : SQLite3